

DevSecOps 2-2-Source-Code-Security

Task 1 - Introduction

Using version control to protect code integrity, confidentiality, and collaboration in modern software development.

Learning and Outcomes

- In this room, we will delve into the fundamentals of source code storage and version control, primarily focusing on the popular version control software, Git. While there are other version control systems, such as VCS or Mercurial, Git has gained widespread adoption in the software development community due to its flexibility, scalability, and powerful features.
- We will begin by understanding what version control is, why it is essential in DevSecOps (Development, Security, and Operations), and explore some examples of version control software. We will then delve into Git, exploring its various commands, workflows, and best practices for securing your codebase.

This room, you will have a solid understanding of source code security, version control with Git, and how to manage your codebase securely and effectively.

Task 2 - Git and Linux

The Story of Git: Created by Linus Torvalds in April 2005 following the sudden revocation of the Linux community's free access to the BitKeeper DVCS. Built to handle the massive scale of the Linux kernel, enabling distributed, location-independent development and efficient multi-contributor merging. Evolved from a quick replacement tool into the world's most popular version control system utilized across diverse industries. Git offers the following benefits:

- Since it is open-source, you can contribute anytime without paying for it.
- Its performance is better than any other version control software since it focuses solely on the contents of files rather than their names.

- Through hashing, Git protects the code and changes history.
- Many organizations are using Git as their primary version control system.

Answer the questions below

| When was Git released?

2005

| What did Linux Kernel use for a DVCS previous to git?

BitKeeper

Task 3 - Version Control Concepts

Version Control: Protects applications from unauthorized access, tampering, and vulnerabilities by maintaining a secure history of edits.

How does it work? Uses a centralized database of changes alongside isolated local working copies, allowing developers to edit files independently before committing improvements.

Distributed and centralized version control : Version control can be centralized or distributed. In centralized version control, there is a single repository, and updates are immediately visible to others upon committing. In contrast, in distributed version control, each user has their repository and must push changes to a central repository for others to see. Git, Mercurial, and Subversion are the most popular version control systems. A distributed version control system like Git or Mercurial offers faster performance, increased error resistance, and advanced capabilities. Because they are more complex than centralized version control systems like Subversion, they can be tricky to learn. In this learning resource, we will focus on Git as it is one of the industry's most widely used and robust version control systems. In the upcoming tasks, we will explore Git's fundamental concepts, commands, and best practices to ensure source code security and efficient collaboration in software development.

Answer the questions below

| What type of version control is Git?

Distributed

Task 4 - Cloud Based Version Control

Introduction: Enabling flexible, real-time collaboration and scalable source code management by storing codebases in the cloud.

Github: Github is cloud-based, and it helps developers store and manage source code from everywhere in the world. It is the oldest of the services. Chris Wanstrath, P.J. Hyett, Tom Preston-Werner, and Scott Chacon founded the company in 2007 and developed Since it was the first open-source code repository on the web, GitHub became the dominant platform. In the present day, developers can host their code and review it.

CI/CD with GitHub Actions: Continuous Integration and Continuous Deployment (CI/CD) is a method to introduce automation into the stages of app development or the Software Development Lifecycle. It introduces ongoing automation and continuous monitoring from the integration phases to delivery and deployment. All these together are what you would call a CI/CD pipeline. Github launched Actions in 2018; it is Github's way of evolving to give developers powerful automation and Continuous Integration & Deployment. It comprises "Workflows", where you can declare (via actions) how to build the code through the development pipeline. You can create activities like push, issue creation, or a new release. For example, to build a container or to deploy a web service.

GitLab: Throughout the DevSecOps path, we'll be using GitLab. Like GitHub, GitLab is an open-core company that provides a DevOps software package that combines the ability to develop, secure, and operate software in a single application. Founded in 2014, GitLab was created by Ukrainian developer Dmitriy Zaporozhets and Dutch developer Sytse Sijbrandij. GitLab was designed to be a set of collaboration tools and a code repository service. GitLab eliminates the need for third-party integration as it includes CI/CD tools by default, making it a convenient all-in-one solution for software development.

CI/CD with GitLab: GitLab's CI/CD functionality allows developers to automate their software delivery pipeline, from building and testing to deploying and monitoring. GitLab uses "runners" - agents that execute jobs - to enable continuous integration and continuous deployment. With GitLab's CI/CD features, developers can create custom workflows, define stages, and specify jobs to be executed in parallel or sequentially. This enables teams to achieve efficient and reliable automation of their development and deployment processes. GitLab also

emphasizes reliability by providing features such as a built-in container registry, built-in continuous deployment to Kubernetes, and GitLab Pages for hosting static websites. Additionally, GitLab allows for creating multiple stable branches beyond the main branch, providing better version control and release management.

In summary, GitLab's built-in CI/CD capabilities, focus on reliability, and all-in-one DevOps platform makes it a robust software development and deployment solution.

Answer the questions below

| When was Github founded?

2007

| Where do Cloud-Based VCS store code?

repositories

Task 5 - Insufficient Credential Hygiene

Introduction: Practicing robust credential hygiene in CI/CD environments to prevent security breaches caused by insecure storage, improper usage, and lack of rotation.

Risk Impact: Attackers highly seek credentials to access valuable resources and enable malicious activities. Engineering environments offer multiple opportunities for attackers to obtain credentials. Human error and knowledge gaps in credential management can increase the risk of credential exposure and compromise of critical resources. Recommendations:

1. Ensure that credentials follow the principle of least privilege from code to deployment.
2. Avoid sharing the same credentials across multiple contexts to maintain accountability and simplify privilege management.
3. Use temporary credentials whenever possible, and establish procedures to rotate static credentials and detect stale credentials periodically.
4. Ensure credentials are only used under predefined conditions, such as limiting usage to a specific IP address or identity.

5. Detect secrets pushed to and stored in code repositories using Integrated Development Environments (IDE) plugins, automatic scanning, and periodic repository commit scans.
6. Use built-in vendor options or third-party tools to prevent secrets from being printed to console outputs during builds and ensure existing outputs do not contain secrets.
7. Verify that secrets are removed from artifacts, such as container image layers and binaries.

Organizations should consider implementing the above best practices to mitigate the risks of insufficient credential hygiene.

Environment Variables and Best Practices: The use of environment variables is an excellent method of promoting credential hygiene. As part of software development and deployment processes, environment variables are commonly used to store and manage configuration information. They store sensitive information, such as API keys, passwords, and other credentials. You can manage environment variables securely by following these best practices:

1. Avoid hardcoding sensitive information in code; use environment variables instead.
2. Regularly review and rotate credentials stored in environment variables.
3. Limit access to environment variables to authorized personnel only.
4. Environment variables should be set according to the principle of least privilege.
5. Implement monitoring and auditing mechanisms to track changes to environment variables.
6. If implementing a secrets manager solution (explained in the next task), review its encryption mechanisms and if it's a good fit for your development environments.

Answer the questions below

| What is a solution to store secrets securely without revealing them?

environment variables

Does using environment variables mean you are free from secrets being compromised?

No

Task 6 - The Git, The Branch and The Ugly

Introduction: Git - Here we will cover basic commands to interact with projects in GitLab. To fetch a project from GitLab, Git uses a command called clone. The syntax looks something like this:

```
git clone <repository_url>
```

It's used to copy (clone) an existing repository (repo) at **another location** in a new directory (**repositories**). Local filesystems or remote machines accessible by supported protocols can host the original repository. Working copies of Git repositories have their history, manage their files, and are isolated from their source repositories. The repository you create on your computer contains all the files and directories of your project. You can think of a branch as different versions of your repository or as an independent development line. Various versions of a repository can be contained in multiple **branches**.

```
git clone -branch
```

The **-branch** argument allows you to specify a branch to clone instead of the default branch the remote is pointing to.

```
git clone -branch name_of_branch <repository_url>
```

The above example would clone only the name_of_branch from the remote Git repository. Branching is a common feature in most Version Control Systems (VCS) that allows you to diverge from the main line of development and work on changes without affecting the main line. Git, a popular VCS, has a unique and powerful branching model that sets it apart. Git branches are lightweight, making branching operations fast and switching between branches quickly. Git encourages frequent branching and merging, even multiple times daily, which is critical to efficient project development.

Understanding Git's Branching Model: To understand Git's branching model, it's essential to know how Git stores its data. Git does not store changesets or differences; instead, it keeps snapshots of the content. When you make a commit in Git, it creates a commit object that includes a pointer to the snapshot of the

content, along with information like author details, commit message, and parent commit(s) pointer(s).

Branching: Git provides the `git branch` command to create, list, and delete branches. As mentioned, branches in Git represent isolated lines of development. When you create a local branch from a remote branch, tracking branches are created automatically by Git, which are local branches linked to remote branches. You can list remote-tracking branches using the `-r` option with the `git branch` command and local and remote branches with the `-a` option.

In Git, local branches exist on your local machine and remote branches are stored in a remote location. Comparing the differences between local and remote branches can be helpful. Here are the steps to do it:

- Update remote-tracking branches by running the command: `git fetch` in the terminal.
- List both local and remote branches using the command: `git branch -a`. This will show the branches with an asterisk indicating the currently checked-out branch.
- To specifically list remote branches, use the following command: `git branch -r`

Adding code: To **add** changes made to a file to the staging area, use the Git `add` command followed by the file name. This tells Git to include the changes made to the file in the next commit.

```
git add <filename>
```

Commit: To save changes to the staging area. This creates a new commit in the Git history with the changes made. Use the commit message to add the description of the changes; very important to track progress.

```
git commit -m "commit message"
```

Push: This will update the remote repository with the changes you made locally. It's the final command to upload the changes from your local repository to the remote. Add the name of the branch you are working on. **Note:** you can pass "origin". "Origin" refers to the repository from which a project was initially cloned.

```
git push <branch name>
```

Answer the questions below

How does Git refer to isolated lines of development?

branches

What term does Git use to refer to the original repository you cloned from?

origin

What command can you use to "copy" the contents in a remote repository?

git clone

Task 7 - USSC Nostromo

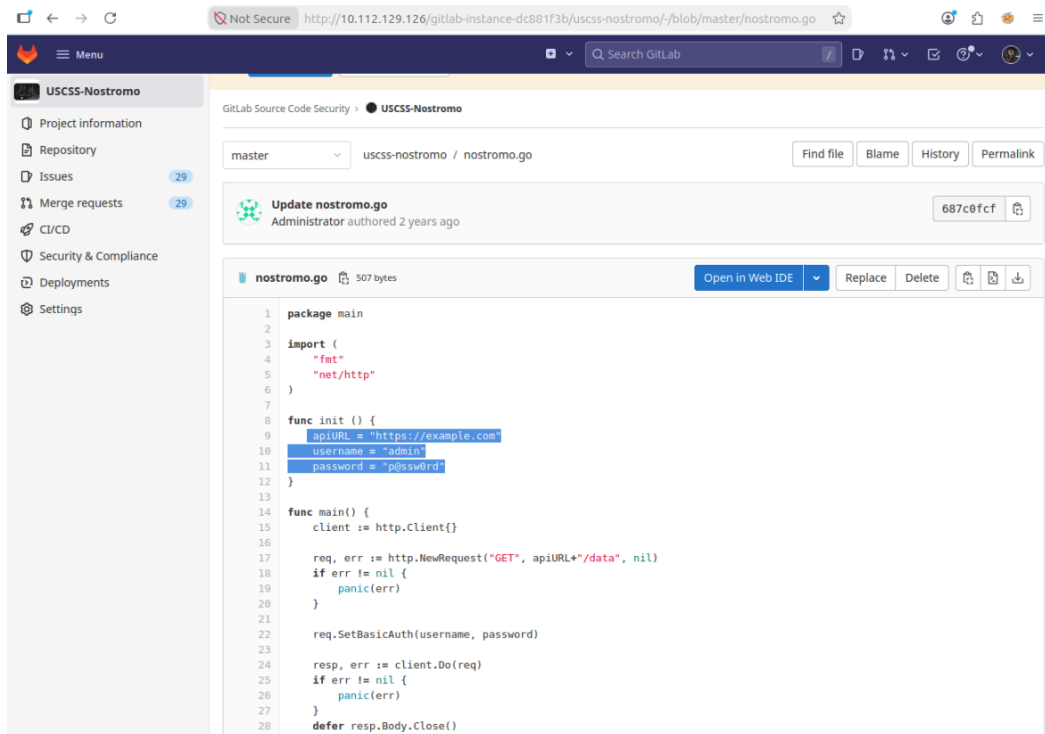
Exercise: In the depths of space, aboard the USSC Nostromo, a team of Developers were on a mission to explore a distant planet. As they journeyed through the void of space, they encountered a strange and terrifying creature - an alien unlike any they had ever seen! As they frantically tried to defend themselves against the monster's attacks, they realized something else was even more dangerous than the alien: the ship's code was riddled with security holes! One of the most alarming issues was that **sensitive authentication credentials** were hard-coded directly into the source code of the ship's software. This meant that anyone who gained access to the code would have access to these critical credentials, putting the entire mission at risk.

Thankfully, the team had DevSecOps Engineers who knew just what to do. They decided to use *environment variables* to store the sensitive information so that the credentials would not be directly visible in the source code. This way, they could prevent unauthorized access to their systems by protecting their credentials. To get started, press the Start Lab Machine button at the top of this task. You may access the VM using the AttackBox or your VPN connection. Then open up your web browser and go to https://LAB_WEB_URL.p.thmlabs.com in order to access the Gitlab server.

Username: TryHackMe

Password: TryHackMe!

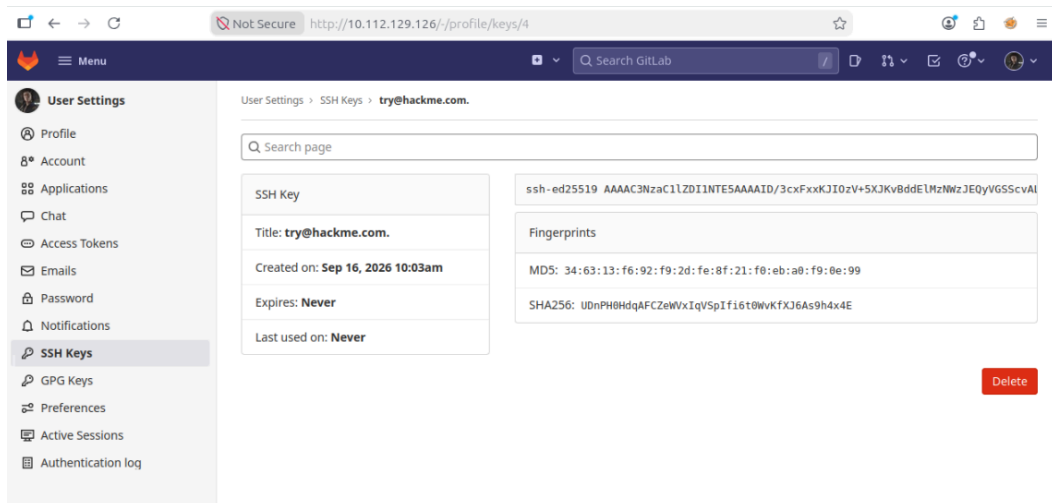
Once logged in, search for the USSC-Nostromo project template in the GitLab search bar.



Open the nostromo repository and create an SSH key using ssh-keygen.

```
root@ip-10-112-75-200: ~
File Edit View Search Terminal Help
root@ip-10-112-75-200:~# ssh-keygen -t ed25519 -C "try@hackme.com".
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:UDnPH0HdqAFCZeWVxIqVSpIfi6t0WvKfXJ6As9h4x4E try@hackme.com.
The key's randomart image is:
+--[ED25519 256]--+
|      .+++++O=O+  |
|      .=OO.=.=.  |
|      . O *. =    |
|      .. B +     |
|      So ..      |
|      o E..      |
|      . Bo....   |
|      o+o+o= .   |
|      o.OO+ o    |
+-----[SHA256]-----+
root@ip-10-112-75-200:~# V
V: command not found
root@ip-10-112-75-200:~# cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAID/3cxFxxKJI0zV+5XJKvBddELmzNWzJEQyVGSScvALM
try@hackme.com.
root@ip-10-112-75-200:~#
```

Add this SSH key to the repository and copy url



Add the IP address to the /etc/hosts file using Vim.

```
root@ip-10-112-74-13: ~
File Edit View Search Terminal Help
127.0.0.1 localhost
10.112.74.13 gitlab.tryhackme.loc
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
```

```
root@ip-10-112-74-13:~# vim /etc/hosts
root@ip-10-112-74-13:~# git clone git@gitlab.tryhackme.loc:gitlab-instance-dc881
f3b/uscss-nostromo.git
Cloning into 'uscss-nostromo'...
The authenticity of host 'gitlab.tryhackme.loc (10.112.74.13)' can't be established.
ED25519 key fingerprint is SHA256:VdIc2rtmuzboXNh0/WdWKLt67PMPQtAyFJ+1ISKbDc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

Set-Up: To interact with a project, you need to add an SSH key to your user to clone the project. While this is not required for this task, you can do so by following these steps:

1. Open a new terminal window.
2. Use the following command to generate a new SSH key:

```
ssh-keygen -t ed25519 -C "try@hackme.com"
```

A prompt will appear to enter a file name to save the key. Press **Enter** to accept the default file name and location. Next, you will be asked to enter a passphrase for the key. You can either enter a passphrase or leave it blank for no passphrase. Note that using a passphrase adds an extra layer of security to your SSH key. Once the key is generated, you can view the public key by typing the following command:

```
cat ~/.ssh/id_ed25519.pub
```

 this will display the public key in the terminal window.

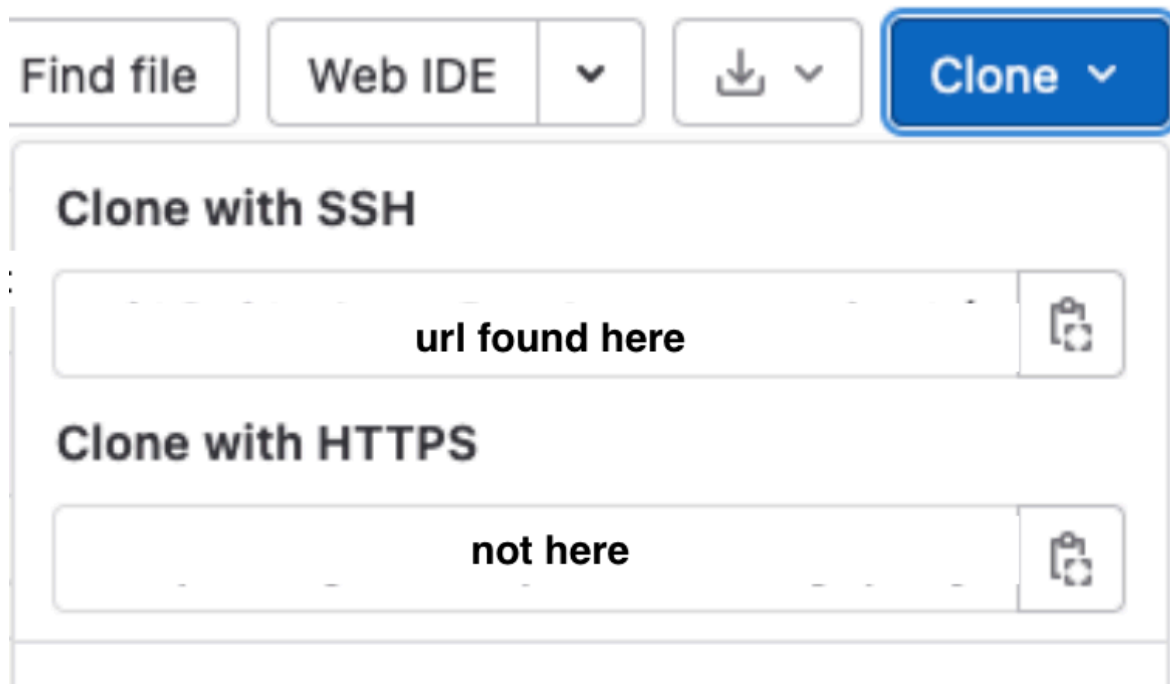
Copy the entire contents of the public key, including the "ssh-ed25519" prefix and email address, to your clipboard. Next, do the following:

1. In the GitLab instance, navigate to your profile settings by clicking "Edit profile".
2. Click on the "SSH Keys" tab (option 3 from the bottom).
3. Paste the contents of your public key into the "Key" field.

4. Give the key a title, such as "My GitLab SSH Key".
5. Click the "Add Key" button to save the SSH key to your GitLab account.

The screenshot shows the GitLab User Settings interface. On the left is a sidebar with 'User Settings' and various options. The 'SSH Keys' option is highlighted. The main panel is titled 'SSH Keys' and contains instructions on how to use SSH keys. Below this, there's a section 'Add an SSH key' which includes a large text area for the key, a 'Title' input field, and an 'Expiration date' input field. A note at the bottom says 'Key titles are publicly visible.' and 'Key becomes invalid on this date.' An 'Add key' button is located at the bottom right of the form.

Cloning the Project: After you become familiar with the project, clone it to your local machine using the following command: `git clone <project-URL>` where **project-URL** is the same URL you used to log in with on your browser. Then enter the login credentials from before. If you want to use the SSH protocol, first add the `gitlab.tryhackme.loc` hostname to your hosts file with the command `echo 'MACHINE_IP gitlab.tryhackme.loc' >> /etc/hosts`. Then get the SSH URL found under the `Clone drop-down`:



Then go into the project with `cd uscss-nostromo` and create a **new branch** to work on by running the following command: `git checkout -b <branch-name-of-you-choice>`

Can you see what's wrong with nostromo.go?

Environment Variables: We mentioned Environment Variables in the previous tasks as part of best practices. They are key-value pairs set in the operating system or shell environment and can be accessed by programs and scripts running in that environment. Environment variables can store various information types, such as **configuration settings**, **file paths**, and **authentication credentials**. They commonly manage software configuration across different environments, such as development, staging, and production. In programming, environment variables are often used to pass information between different parts of an application or system or to configure a program's behaviour based on its environment.

os.getenv

It would be best if you had spotted the plaintext credentials in nostromo.go by now. To make changes to the project, open the uscss-nostromo/nostromo.go file and replace the hard-coded credentials with environment variables. To implement environment variables, we will make use of `os.getenv`. **Os.getenv** is a function used to retrieve the values of environment variables. The **os.getenv** function takes

a single argument: the name of the environment variable you want to retrieve. If the environment variable is found, the function returns its value as a string. The function returns None (or nil in some programming languages) if the environment variable is not found. The package we need to import to use `os.getenv` is `os`. We just need to add `import ("os")` as shown below:

```
import (  
    "fmt"  
    "net/http"  
    "os"  
)
```

To implement `os.getenv`, you can replace these lines:

```
func init () {  
    apiURL = "https://example.com"  
    username = "admin"  
    password = "p@ssw0rd"  
}
```

With these lines:

```
func init () {  
    apiURL = "https://example.com"  
    username = os.Getenv("GITLAB_USERNAME")  
    password = os.Getenv("GITLAB_PASSWORD")  
}
```

Once you've made your changes, add, commit, and push them to the branch using the following command:

```
git commit -a -m "Fixed credential hygiene by using environment variables"
```

To push your changes to the branch, use the following command:

```
git push -u origin <branch-name-chosen-earlier>
```

Verify that the changes you made in the branch are now in the main branch by checking the `uscss-nostromo/nostromo.go` file for your updated credentials. In this updated codebase, we have the following:

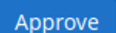
1. Used the `os.Getenv()` function to read the `NOSTROMO_API_URL`, `NOSTROMO_USERNAME`, and `NOSTROMO_PASSWORD` environment variables, which can be managed using GitLab's secrets management feature.
2. Since `os.Getenv` retrieves sensitive information such as database credentials, the `nostromo.go` codebase is more secure and easier to maintain. If the credentials need to be updated or rotated, they can be updated in the environment variables without modifying the code directly.

Fixed credential hygiene by using environment variables

 Open Lt Ellen Ripley requested to merge `testing` into `master` just now

Overview 0 Commits 1 Changes 1

 Checking pipeline status. ?

  Approval is optional ?

Don't merge the changes yet, we still haven't finished securing the Nostromo API. Continue to the next task to retrieve the final flag!

Answer the questions below

What is the name of the package that you need to import to make use of `os.getenv`?

os

What is the hidden flag?

THM-3LL3N-RIPL3Y

DEVSEC-113-Sec...	uscss-nostromo /	+	History	Find file	Web IDE	↓	Clone
 THM-3LL3N-R1PL3Y Administrator authored 5 months ago							7eeb1a68 
Name	Last commit	Last update					
 README.md	README.md	5 months ago					
 nostromo.go	THM-3LL3N-R1PL3Y	5 months ago					

Task 8 - Secret Management

Introduction: As seen in the previous task, environment variables are essential to securing digital authentication credentials, such as passwords, keys, APIs, and tokens, used in applications, services, privileged accounts, and other sensitive parts of the IT ecosystem. However, we need to implement a **secret management** solution to use these environment variables correctly. Inadequate secrets management can result in security gaps and auditing challenges, leading to potential security breaches. In this tutorial, we will explore how to manage secrets with GitLab.

Managing Secrets in GitLab

Once you have added the environment variables to your GitLab project, you can use GitLab's secret management functionality to store and access these variables securely. Here's how you can do it:

1. Go to your GitLab project and navigate to the "Settings" page.
2. Click on the "CI/CD" tab and then click on "Variables".
3. Click on "Add variable" and enter the name and value of your environment variable. Make sure to mark the variable as "Protected".
4. Click on "Add variable" to save the variable.
5. Now, in your nostromo.go file, you can use the syntax `os.Getenv("variable_name")` to access the value of the environment variable.

For example, we have added variables named GITLAB_USERNAME and GITLAB_PASSWORD; you can access them in nostromo.go like this:

```
func init () {
  apiURL = "https://example.com"
  username = os.Getenv("GITLAB_USERNAME")
  password = os.Getenv("GITLAB_PASSWORD")
}
```

Now you can securely store the credentials and access them using environment variables.

Bonus

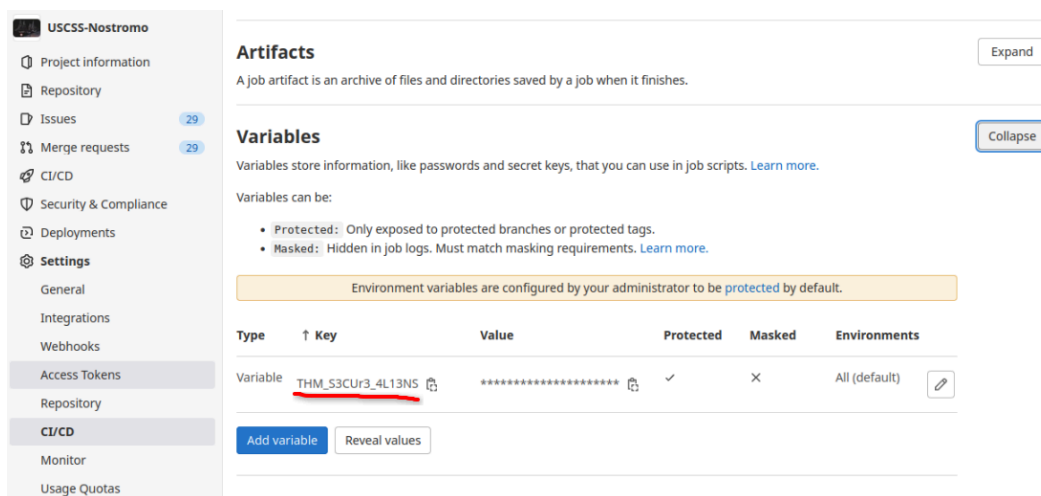
Have you noticed there is a `.gitlab-ci.yml` file? But there are no pipelines, workflows or "status checks"? To research what that file is for, check out: https://docs.gitlab.com/ee/ci/yaml/gitlab_ci_yaml.html (opens in new tab). Although is out of scope for this room, keep your eyes peeled for the next room in the pathway: "Build System Security".

Can you figure out what the contents of `.gitlab-ci.yml` means?

Answer the questions below

What is the hidden flag?

THM_S3CUr3_4L13NS



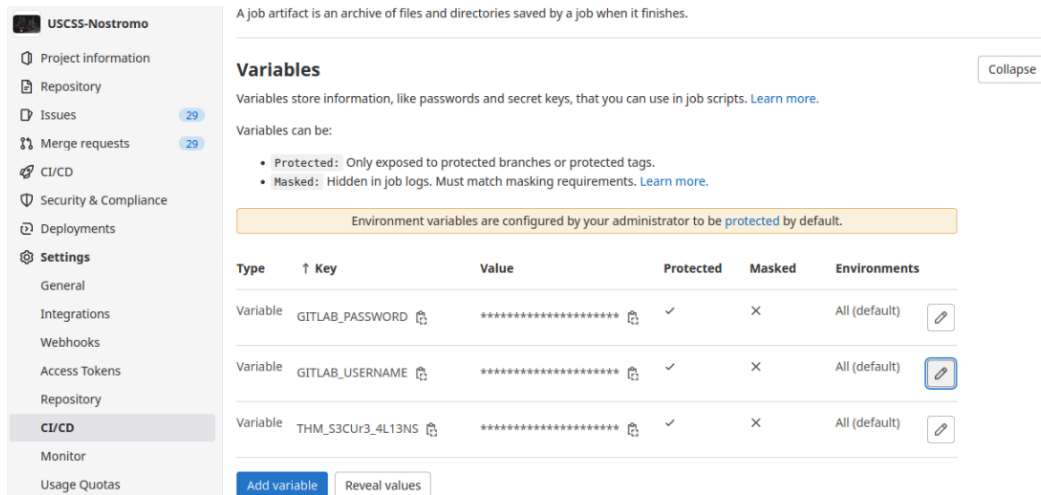
The screenshot shows the GitLab interface for the project 'USCSS-Nostromo'. The left sidebar contains navigation links: Project information, Repository, Issues (29), Merge requests (29), CI/CD, Security & Compliance, Deployments, Settings, General, Integrations, Webhooks, Access Tokens, Repository, CI/CD, Monitor, and Usage Quotas. The main content area is titled 'Variables' and includes a description: 'Variables store information, like passwords and secret keys, that you can use in job scripts. Learn more.' It lists two types of variables: Protected (Only exposed to protected branches or protected tags) and Masked (Hidden in job logs. Must match masking requirements. Learn more.). A yellow banner states: 'Environment variables are configured by your administrator to be protected by default.' Below this is a table of environment variables:

Type	Key	Value	Protected	Masked	Environments
Variable	THM_S3CUr3_4L13NS	*****	✓	✗	All (default)

At the bottom of the table are buttons for 'Add variable' and 'Reveal values'.

What do you need to keep source code secure besides environment variables?

secret management



The screenshot shows the GitLab CI/CD settings page for a project named 'USCSS-Nostromo'. The left sidebar contains a navigation menu with options like Project information, Repository, Issues, Merge requests, CI/CD, Security & Compliance, Deployments, Settings, General, Integrations, Webhooks, Access Tokens, Repository, CI/CD (selected), Monitor, and Usage Quotas. The main content area is titled 'Variables' and explains that variables store information like passwords and secret keys. It lists two types of variables: Protected (exposed to protected branches or tags) and Masked (hidden in job logs). A note states that environment variables are configured by the administrator to be protected by default. Below this is a table of environment variables.

Type	Key	Value	Protected	Masked	Environments
Variable	GITLAB_PASSWORD	*****	✓	✗	All (default)
Variable	GITLAB_USERNAME	*****	✓	✗	All (default)
Variable	THM_53CUIr3_4L13NS	*****	✓	✗	All (default)

Buttons at the bottom: Add variable, Reveal values.

then merge

Which file handles the configuration to run CI/CD jobs?

.gitlab-ci.yml